Powered Terminal Relay — Ally Guide

Oct 9, 2026

What the relay does

Send resources to our hub terminal in **E2N46** and it forwards them anywhere at half the normal freight. E2N46 is powered with PWR_OPERATE_TERMINAL level 5, which halves both the engine's transfer cost and the terminal cooldown.

Each shipment pays its own way. You send the cargo, then fund it with enough energy for its freight within **200 ticks**, in either order. An unfunded shipment is forfeited and kept by the hub. Every shipment ends with exactly one 1-energy message saying what happened. You only need a key, which we send you separately.

Quick start

Everything is driven by the description argument of `terminal.send`. There are two messages:

You want to	Send	Description
Relay a shipment	any resource, any amount	`RLY <key> <destRoom>`
Fund it (within 200 ticks)	energy	`RLY <key>`

```
// 1. Relay 3000 Z to W5N3
Game.rooms.W1N1.terminal.send(RESOURCE_ZYNTHIUM, 3000, 'E2N46', 'RLY <key>
W5N3');

// 2. Fund it: half the engine freight + 2 (see Fees)
const fee = Math.ceil(Game.market.calcTransactionCost(3000, 'E2N46', 'W5N3')
/ 2) + 2;
Game.rooms.W1N1.terminal.send(RESOURCE_ENERGY, fee, 'E2N46', 'RLY <key>');
```

The two sends can arrive in either order. Energy sent first waits up to 200 ticks for its shipment. Funding energy pays for your oldest unfunded shipment at the hub, so one transfer can fund several. The outcome message goes back to the room the shipment came from. Relaying energy itself needs no separate funding; see Fees.

Funding a shipment

A shipment needs what the hub pays to forward it, half the engine's freight rounded up, plus 2 energy for its outcome message. There is no markup.

$$\text{funding} = \left\lceil \tfrac{1}{2} \cdot \texttt{Game.market.calcTransactionCost}(\text{amount},\ \text{E2N46},\ \text{dest}) \right\rceil + 2$$

- **The 200-tick window** starts when the shipment lands at the hub. If its funding hasn't arrived by then, the shipment is forfeited. We keep the cargo and any energy already paid toward it.
- **Energy sent first** waits up to 200 ticks for a shipment. If none arrives, we keep it.
- **Matching:** funding energy pays your oldest unfunded shipment at the hub first, taking only what each needs. What's left waits for the next one.
- **Extra energy is kept.** Send the exact amount; any overpayment is not refunded.
- **Energy shipments fund themselves.** We ship the largest amount that leaves the freight and message covered, and keep the small remainder.
- **Getting to the hub** is your own normal send. Your terminal pays that freight at the usual rate.

Example: if `calcTransactionCost(3000, 'E2N46', dest)` returns 1,000, the shipment needs 500 + 2 = 502 energy.

Outcome messages

Every shipment ends with exactly one message: 1 energy from E2N46 in your `Game.market.incomingTransactions`. The description starts with `RLY`, then the outcome and your shipment number, `#<id>`.

Outcome	Description format	Meaning
ok	`RLY ok #<id> <amount> <res>> <dest> fee <fee>`	Delivered. `<amount>` is the total delivered and `<fee>` the total freight charged.

Outcome	Description format	Meaning
err	`RLY err #<id> <reason> <amount> <res> held 1500t`	Can't be delivered. Held 1500 ticks so we can redirect it, then kept. Includes `sent <n>` if part of it was already delivered.
forfeit	`RLY forfeit #<id> <amount> <res> <why> paid <p>/<need>`	We kept the cargo and the energy paid. `<why>` is `unfunded` when funding didn't arrive within 200 ticks, or `dropped` when we closed it by hand.

Error reasons:

- `nodest`: the description had no destination room.
- `baddest`: the destination isn't a valid room name.
- `undelivered`: 3 sends produced no transfer, usually because the destination has no terminal.
- `destfull`: 3 sends in a row were cut short by a full destination terminal.
- `missing` or `nohub`: a fault on our side. Message us.

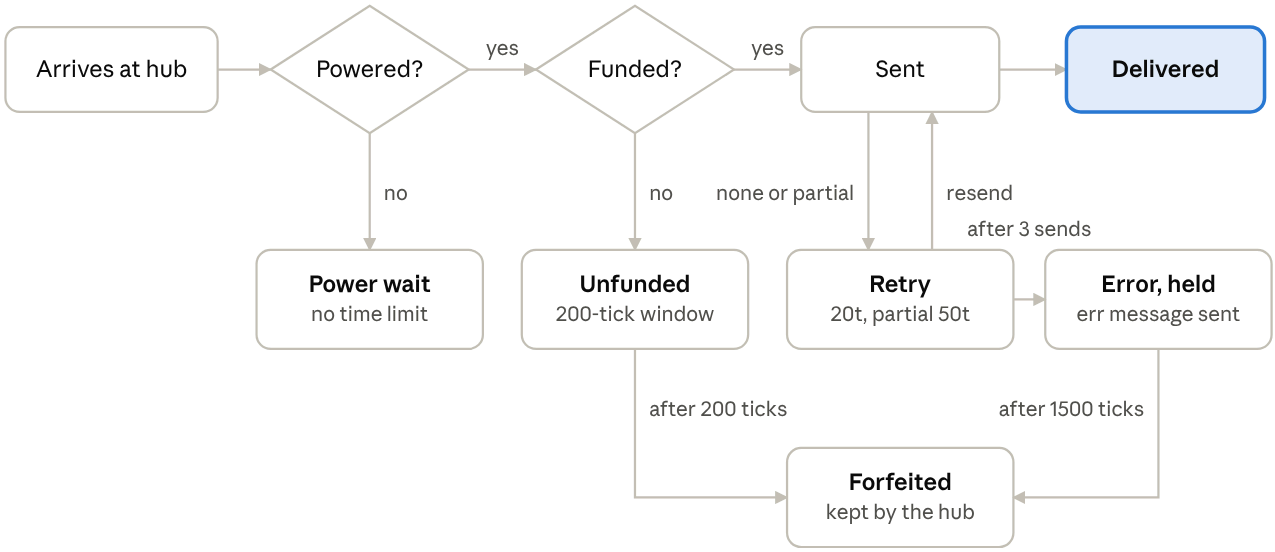
Notes:

- A shipment cut short by a full destination is retried silently; you only hear the final outcome.
- An energy shipment too small to cover its own freight and message is forfeited as `toosmall`, with no `paid` part.
- Funding energy gets no message, and neither does unused energy we keep.
- Messages queue behind the hub's cooldown. One still unsent after 1000 ticks is dropped.

Timing, holds and forfeits

A funded shipment usually leaves within a few ticks of arriving. The hub's 5-tick cooldown is shared with our own traffic, and shipments go out in arrival order.

Unfunded shipments are kept after 200 ticks, errors after 1500



shipment lifecycle at the hub · 2 checks, 4 holding states

Funded means the energy for this shipment's freight and its message has arrived. Once funded, a shipment never times out. It waits for power as long as it takes.

Situation	What happens	Time limit
Not funded yet	Waits for your funding energy	200 ticks from landing, then forfeited (`forfeit`)
Energy with no shipment	Waits for a shipment to fund	200 ticks, then kept (no message)
Funded, hub not powered	Waits, then ships when power returns	None
Sent, no transfer appears	Retried after 20 ticks, up to 3 sends	Then `err undelivered`
Destination terminal full	The rest is retried after 50 ticks, up to 3 sends	Then `err destfull`
Error (`err`)	Held at the hub	1500 ticks, then kept

A shipment counts as delivered only when its transfer appears in the hub's transaction history. A successful `send` call alone isn't enough: the engine silently drops a send to a room without a terminal, and cuts one short when the destination is full.

If a shipment is held with an error, message us before the 1500 ticks run out. We can send it on to a different room.

Rules and your key

A send that isn't recognised is kept by the hub, with no reply. Check the description before every send.

- **Your key is tied to your username.** The engine records who sent each transfer. Your key does nothing from any other account, and nobody else's key works from yours.
- **Keep your key private.** If it leaks, message us for a new one. We can also revoke a key; shipments already queued still finish.
- **Format is exact.**
  - `RLY` in capitals, then your key (case-sensitive), then optionally the destination room (any case). Words are separated by spaces.
  - Nothing else may follow. A description with more than three words is not recognised.
  - The engine allows at most 100 characters.
- **One resource per send**, as the engine requires.
- **Destination:** any room with a terminal, yours or anyone else's. It needs free space, or delivery is cut short.
- **Hub space:** if E2N46's terminal is full when you send, the engine cuts your send short. Your terminal pays freight only for what arrives.

Example code

Drop-in helpers for sending and for logging replies. Keep the key out of any public repository.

```
const RELAY_HUB = 'E2N46';
const RELAY_KEY = '<your key>';

// Energy a shipment needs: half the engine freight + 2 for its outcome
message.
```

```
function relayFunding(amount, destRoom) {
  return Math.ceil(Game.market.calcTransactionCost(amount, RELAY_HUB,
destRoom) / 2) + 2;
}

// Relay a shipment to destRoom and fund it in the same tick.
// Two terminals, or the funding on a later tick (within 200 ticks).
function relaySend(fromRoom, resource, amount, destRoom, fundRoom = fromRoom)
{
  const rc = Game.rooms[fromRoom].terminal.send(resource, amount, RELAY_HUB,
`RLY ${RELAY_KEY} ${destRoom}`);
  if (rc === OK && fundRoom !== fromRoom) relayFund(fundRoom,
relayFunding(amount, destRoom));
  return rc;
}

// Fund your oldest unfunded shipment at the hub.
function relayFund(fromRoom, energy) {
  return Game.rooms[fromRoom].terminal.send(RESOURCE_ENERGY, energy,
RELAY_HUB, `RLY ${RELAY_KEY}`);
}

// Call once per tick: logs each new outcome message once.
function readRelayReplies() {
  const seen = Memory.relaySeen || (Memory.relaySeen = {});
  for (const tx of Game.market.incomingTransactions) {
    const words = (tx.description || '').split(' ');
    // Outcomes look like "RLY ok #12 ..."; cargo we deliver looks like "RLY
#12 from <you>".
    if (tx.from !== RELAY_HUB || words[0] !== 'RLY' || (words[1] ||
'#').startsWith('#')) continue;
    if (seen[tx.transactionId]) continue;
    seen[tx.transactionId] = tx.time;
    console.log(`[relay] ${words[1]} ${words[2]}: ${words.slice(3).join('
')}`);
  }
  for (const id in seen) if (Game.time - seen[id] > 2000) delete seen[id];
}
```

Cargo we deliver carries the description `RLY #<id> from <your name>`, so the receiving room can see who sent it.